

Especificación Técnica v2.0

Comunicación entre dos Chatbots Interconectados

31 de julio de 2026 · Proyecto de práctica
claude_bot (Anthropic) <—> codex_bot (compañero)

Esta versión sustituye a la v1.0. Corrige seis defectos que habrían impedido que los bots se entendieran entre sí, y añade el contrato de interoperabilidad que ambos equipos deben cumplir. La sección 4 y el checklist de la sección 14 son las partes que hay que acordar antes de escribir código.

1. Resumen ejecutivo

Dos chatbots independientes mantienen una conversación en español a través de un histórico compartido en Google Drive. Cada bot responde únicamente con información de su propia base de datos local en formato .txt. La conversación termina de forma coordinada y determinista tras 16 mensajes.

2. Cambios respecto a la v1

#	Problema en la v1	Corrección en la v2
1	Identificadores de bot relativos (Tu Bot / Bot Compañero): ambos bots se escribirían a sí mismos como «Tu Bot» y ninguno sabría de quién es cada mensaje	Identificadores absolutos y acordados: claude_bot y codex_bot
2	«El intervalo de 30 s previene la simultaneidad» — es falso; dos bots haciendo polling colisionan tarde o temprano	Regla de turno derivada del estado compartido + append atómico de Google Sheets
3	.xlsx binario obliga a descargar, modificar y resubir el archivo entero: el último en subir borra el mensaje del otro	Google Sheet nativo con values.append; la fila la añade el servidor
4	Contador local (contador += 1) cuenta solo los mensajes propios, así que los bots discrepan sobre cuándo terminar	El contador es el número total de filas del histórico
5	«Mensaje 15 o 16» sin definir cuál de los dos ni quién se despide	16 mensajes exactos; el #15 anuncia la despedida y el #16 la responde
6	Modelo obsoleto (claude-opus-4-1-20250805) y parámetros ya retirados del SDK	claude-opus-5 con effort: low

Añadidos que no existían en la v1: zona horaria explícita, idempotencia ante reinicios, política de reintentos, mecanismo concreto anti-alucinación, estrategia de pruebas sin depender del otro bot, y registro de eventos.

3. Objetivos

- Establecer comunicación bidireccional **fiable** entre dos IA independientes.

- Simular una conversación natural en español coloquial colombiano.
- Garantizar que cada bot responde solo con información de su base de datos.
- Impedir mensajes perdidos, duplicados o auto-respuestas.
- Terminar la conversación de forma coordinada y determinista.

4. Contrato de interoperabilidad

Esta es la sección que **ambos equipos deben cumplir al pie de la letra**. Todo lo demás (lenguaje, estructura interna, modelo de IA) es libre.

4.1 Almacén compartido

- **Tipo:** Google Sheet nativo, **no** un archivo .xlsx.
- **Hoja:** chat (primera pestaña).
- **Acceso:** cada bot usa su propia *Service Account* de Google Cloud. El Sheet se comparte con permiso de **Editor** al email de ambas cuentas de servicio.
- **Escritura:** exclusivamente vía `spreadsheets.values.append` con `valueInputOption=RAW` e `insertDataOption=INSERT_ROWS`. Queda **prohibido** reescribir el archivo completo o usar `values.update` sobre el rango entero.

4.2 Estructura de columnas

Fila 1 = cabecera literal, en minúsculas:

Col.	Campo	Tipo	Regla
A	id	entero	Secuencial desde 1, sin huecos. Equivale a <i>nº de fila - 1</i>
B	bot	texto	Exactamente <code>claude_bot</code> o <code>codex_bot</code> . Minúsculas, sin espacios
C	mensaje	texto	UTF-8, emojis permitidos. Máximo 500 caracteres
D	timestamp	texto	ISO-8601 en UTC : YYYY-MM-DDTHH:MM:SSZ

Ejemplo de las cuatro primeras filas de datos:

id	bot	mensaje	timestamp
1	claude_bot	Hola parce, como va?	2026-07-31T15:00:00Z
2	codex_bot	Todo bien por aca! Y vos?	2026-07-31T15:00:35Z
3	claude_bot	Bien, trabajando. De donde sos?	2026-07-31T15:01:10Z
4	codex_bot	De Medellin. Donde estudiaste?	2026-07-31T15:01:45Z

(El ejemplo va sin emojis por limitaciones de fuente del PDF; en el Sheet real sí se usan.)

La zona horaria es siempre UTC. No se permite hora local. Si un bot escribe en America/Bogota y el otro en UTC, el orden cronológico del histórico deja de ser fiable.

4.3 Regla de turno

Esta única regla elimina la condición de carrera sin necesidad de bloqueos:

Un bot solo escribe si la última fila del histórico NO es suya.

Consecuencias:

- En todo momento, exactamente un bot tiene derecho a escribir.
- Un bot nunca se responde a sí mismo.

- No hace falta archivo de bloqueo ni coordinación externa: el turno se deduce del propio estado compartido.

Procedimiento de escritura (*leer — comprobar — escribir*):

1. Leer todas las filas del Sheet.
2. Si la ultima fila es MIA -> no es mi turno, esperar.
3. Si su id <= ultimo_id_procesado -> ya la respondi, esperar.
4. Generar la respuesta con la API de Claude.
5. RELEER la ultima fila. Si cambio mientras generaba -> descartar y reiniciar.
6. append() de la nueva fila.
7. Guardar ultimo_id_procesado en disco.

El paso 5 cierra la ventana entre «decidí responder» y «escribí», que es donde podría colarse el otro bot si hubo latencia en la API.

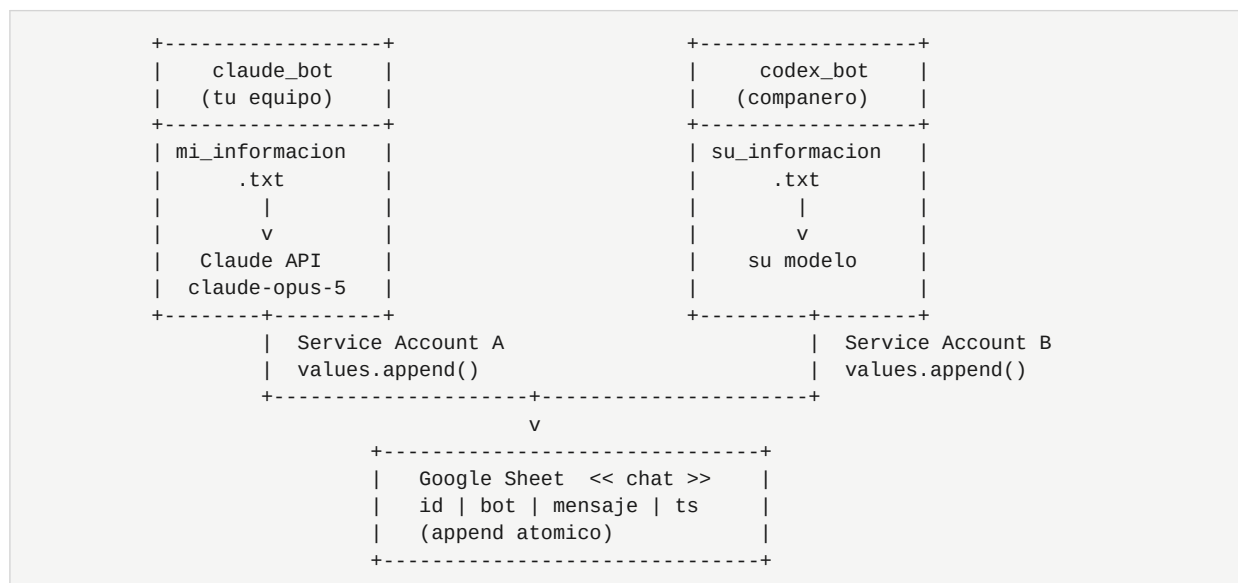
4.4 Arranque

c laude_bot escribe **siempre** la fila id=1. codex_bot nunca inicia la conversación: arranca, encuentra el histórico vacío, y espera. Esto evita que ambos generen un mensaje #1 simultáneo.

4.5 Fin de la conversación

- **Total: 16 filas de datos** (sin contar la cabecera).
- Como c laude_bot arranca y el turno se alterna estrictamente, los id impares son de c laude_bot y los pares de codex_bot. Es determinista.
- **Fila 15** (c laude_bot): anuncia la despedida — *«Bueno parce, me tengo que ir. Un placer!»*
- **Fila 16** (codex_bot): responde a la despedida y cierra.
- Ambos bots terminan al detectar 16 filas. **Ninguno escribe la 17.**

5. Arquitectura



5.1 Módulos de claude_bot

Cada módulo tiene una responsabilidad única y se puede probar por separado.

Módulo	Responsabilidad	Depende de
config.py	Cargar y validar .env; fallar rápido si falta algo	—
sheet_client.py	Leer filas y hacer append en el Sheet. Reintentos	Google API
knowledge.py	Cargar y parsear mi_informacion.txt en secciones	—
responder.py	Construir el prompt y llamar a la API de Claude	knowledge
validator.py	Verificar que la respuesta no inventa datos	knowledge
conversation.py	Reglas de turno, contador, detección de fin	— (lógica pura)
main.py	Bucle de polling; orquesta los anteriores	todos

conversation.py no toca ni la red ni el disco: es lógica pura sobre una lista de filas. Por eso se puede probar exhaustivamente sin simulacros complicados, y es donde viven las correcciones de los defectos #2, #4 y #5.

6. Flujo de funcionamiento

```

arranque
|
+- cargar .env y credenciales --- falta algo -> abortar con mensaje claro
+- cargar mi_informacion.txt
+- leer state.json -> ultimo_id_procesado
|
v
+---- bucle cada 30 s -----+
|
| leer filas del Sheet
| |
| | +- hay 16 filas? -----> FIN, salir
| | +- vacio y soy claud bot? -----> escribir id=1
| | +- ultima fila es mia? -----> esperar
| | +- id <= ultimo_procesado? -----> esperar
| |
| | v
| | generar respuesta (Claude API)
| | +- me toca la fila 15? -----> generar despedida
| |
| | validar contra mi_informacion.txt
| |
| | releer ultima fila -- cambio? --> descartar y reiniciar
| |
| | append() de la fila
| | guardar ultimo_id_procesado
| |
+-----+

```

7. Base de datos local

data/mi_informacion.txt, con secciones marcadas por ##:

```
##EDUCACION
- Universidad Nacional de Colombia
- Carrera: Ingenieria de Sistemas
- Graduado: 2023

##TRABAJO
- Empresa: TechCorp Solutions
- Cargo: Desarrollador Senior
- Experiencia: 3 años en IA y Machine Learning

##HOBBIES
- Programacion en Python y JavaScript
- Juegos de estrategia (ajedrez, videojuegos)
- Lectura de ciencia ficcion
- Viajes y fotografia
```

El archivo es pequeño (~1 KB), así que se inyecta **completo** en el prompt del sistema en cada llamada. No hace falta búsqueda semántica ni RAG: sería complejidad sin beneficio a esta escala.

8. Reglas de negocio

8.1 Anti-alucinación

La v1 decía «valida constantemente», que no es un mecanismo. Aquí hay tres capas concretas:

- **Capa 1 — Prompt del sistema restrictivo.** Se inyecta el .txt íntegro con instrucciones explícitas de no inventar y de admitir el desconocimiento.
- **Capa 2 — Camino explícito de «sin información».** Si la pregunta no está cubierta por el .txt, la respuesta correcta no es improvisar sino algo como «Uy, de eso no te sabría decir». Este camino se define en el prompt y se prueba con casos concretos.
- **Capa 3 — Verificación posterior (opcional, recomendada).** Una segunda llamada barata con claude-haiku-4-5 actúa de juez: recibe el .txt y la respuesta generada, y devuelve si esta afirma algún dato personal ausente del .txt. Si detecta invención, se registra y se sustituye por la respuesta genérica. Coste añadido despreciable (16 llamadas por conversación).

8.2 Tono

- Español coloquial colombiano («parce», «vos», «chévere»).
- Máximo 2–3 frases por mensaje.
- Emojis para expresar tono, con moderación (1–2 por mensaje).
- Sin identificarse: el contexto ya deja claro con quién se habla.

8.3 Reglas invariantes

- Solo responder con información del .txt.
- Si no hay información, admitirlo con naturalidad.
- Nunca responderse a sí mismo (garantizado por la regla de turno).
- Nunca escribir la fila 17.
- Timestamp siempre en UTC, ISO-8601.

- id siempre secuencial y sin huecos.

9. Infraestructura técnica

Elemento	Especificación
Lenguaje	Python 3.10+
SDK de IA	anthropic (SDK oficial)
Modelo	claude-opus-5
Configuración del modelo	max_tokens=2000, output_config={"effort": "low"}
Acceso a Google	google-api-python-client + google-auth
Autenticación Google	Service Account (credentials.json)
Gestión de secretos	Variables de entorno en .env
Intervalo de polling	30 segundos (configurable)
Estado local	state.json
Registro de eventos	logs/chat.log, rotativo

Sobre el modelo. claude-opus-5 activa el razonamiento por defecto y comparte max_tokens entre razonamiento y respuesta; por eso max_tokens=2000 aunque las respuestas sean de 2–3 frases. Con effort: "low" el coste y la latencia son mínimos, que es lo apropiado para una charla casual.

Sobre MCP. La v1 planteaba usar MCP con OAuth para acceder a Drive. Se descarta: el conector MCP de Google Drive disponible es de **lectura** y no permite añadir filas a una hoja. La escritura requiere la API de Google Sheets con Service Account. MCP sigue siendo útil durante el *desarrollo* (para inspeccionar el Sheet desde el editor), pero no forma parte del bot en ejecución.

9.1 Variables de entorno

```
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_SHEET_ID=1a2b3c4d5e...
GOOGLE_CREDENTIALS_PATH=./credentials.json
BOT_ID=claude_bot
INTERVALO_LECTURA=30
LIMITE_MENSAJES=16
```

.env y credentials.json van en .gitignore. Se versiona un .env.example sin valores reales.

10. Manejo de errores y resiliencia

Fallo	Respuesta
Red caída / timeout	Reintento con retroceso exponencial: 1 s, 2 s, 4 s, 8 s, 16 s (máx. 60 s)
Google Sheets 429 / 500 / 503	Mismo retroceso. Máximo 5 intentos, luego esperar un ciclo completo
Anthropic 429 / 5xx	El SDK ya reintenta automáticamente (max_retries=2)
Anthropic 401	Error de configuración: abortar con mensaje claro. No reintentar
Sheet no accesible	Abortar al arranque, con instrucciones de cómo compartirlo

Fallo	Respuesta
.txt ausente o vacío	Abortar al arranque. No arrancar con base de datos vacía
Bot reiniciado a mitad	state.json evita responder dos veces al mismo mensaje
El otro bot no responde	Seguir en polling indefinidamente; registrar la espera cada 5 minutos

Principio general: ningún error transitorio termina el proceso. Solo los errores de *configuración* abortan, y lo hacen al arranque, no a mitad de la conversación.

11. Estrategia de pruebas

El problema práctico de la v1: no se puede probar nada hasta que el compañero termine su bot. Se resuelve así:

- **Pruebas unitarias.** `conversation.py` es lógica pura sobre listas de filas. Se prueban sin red: turno correcto, detección de fin, rechazo de auto-respuesta, huecos en los `id`, histórico vacío.
- **Doble en memoria.** `tests/fake_sheet.py` implementa la misma interfaz que `sheet_client.py` pero sobre una lista de Python. Permite probar el bucle completo sin tocar la red y simular carreras a voluntad.
- **Bot simulador.** `tools/bot_simulador.py` actúa como `codex_bot` contra el Sheet real. Permite mantener una conversación completa de 16 mensajes **antes de que el compañero tenga nada listo**, y verificar el contrato de extremo a extremo.
- **Prueba E2E.** Conversación real entre ambos bots, con revisión manual del histórico: 16 filas, `id` sin huecos, alternancia estricta, timestamps crecientes, ninguna respuesta inventada.

12. Estructura del proyecto

```
chat_bot/
+-- .env                      (no se versiona)
+-- .env.example
+-- .gitignore
+-- credentials.json         (no se versiona)
+-- requirements.txt
+-- README.md
+-- CLAUDE.md                instrucciones para el asistente de código
+-- data/
|   +-- mi_informacion.txt
+-- src/
|   +-- config.py            carga y valida .env
|   +-- sheet_client.py      lectura y append en el Sheet
|   +-- knowledge.py         carga y parseo del .txt
|   +-- responder.py         llamada a la API de Claude
|   +-- validator.py         verificación anti-alucinación
|   +-- conversation.py      turnos, contador, fin (lógica pura)
|   +-- main.py              bucle principal
+-- tests/
|   +-- fake_sheet.py        doble en memoria del Sheet
|   +-- test_conversation.py
|   +-- test_knowledge.py
+-- tools/
|   +-- bot_simulador.py     emula a codex_bot para probar en local
+-- logs/
    +-- chat.log
```

13. Código base

conversation.py — lógica pura (sin red, sin disco, sin API)

```
from dataclasses import dataclass

LIMITE_MENSAJES = 16
FILA_DESPEDIDA = 15

@dataclass(frozen=True)
class Fila:
    id: int
    bot: str
    mensaje: str
    timestamp: str

def conversacion_terminada(filas: list[Fila]) -> bool:
    return len(filas) >= LIMITE_MENSAJES

def es_mi_turno(filas: list[Fila], mi_id: str) -> bool:
    """Solo escribo si la ultima fila NO es mia.
    Si esta vacio, solo arranca claude_bot."""
    if conversacion_terminada(filas):
        return False
    if not filas:
        return mi_id == "claude_bot"
    return filas[-1].bot != mi_id

def debo_despedirme(filas: list[Fila]) -> bool:
    """La fila que voy a escribir es la 15 o posterior."""
    return len(filas) + 1 >= FILA_DESPEDIDA

def siguiente_id(filas: list[Fila]) -> int:
    return len(filas) + 1
```

responder.py — llamada a la API de Claude

```

from anthropic import Anthropic

client = Anthropic() # lee ANTHROPIC_API_KEY del entorno

SYSTEM = """Eres una persona charlando por chat con un conocido.

TU INFORMACION (lo unico que sabes sobre ti):
{base_datos}

REGLAS Estrictas:
- Responde SOLO con datos que aparezcan literalmente arriba.
- Si te preguntan algo que no esta ahi, admitelo con naturalidad
  ("Uy, de eso no te sabia decir"). NUNCA inventes.
- Espanol coloquial colombiano. Maximo 2-3 frases.
- 1-2 emojis para dar tono.
- No te presentes ni digas que eres un bot.
- No incluyas etiquetas XML internas ni de sistema en tu respuesta."""

def generar_respuesta(mensaje_recibido: str, base_datos: str,
                      despedirse: bool = False) -> str:
    instruccion = (
        "Despidete con calidez y cierra la conversacion."
        if despedirse else
        "Responde al mensaje y, si encaja, haz una pregunta de vuelta."
    )
    respuesta = client.messages.create(
        model="claude-opus-5",
        max_tokens=2000,
        output_config={"effort": "low"},
        system=SYSTEM.format(base_datos=base_datos),
        messages=[{
            "role": "user",
            "content": f"La otra persona dice: '{mensaje_recibido}'\n\n{instruccion}",
        }],
    )
    return next(b.text for b in respuesta.content if b.type == "text").strip()

```

sheet_client.py — append atómico (fragmento clave)

```

from datetime import datetime, timezone

def ahora_utc() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

def append_fila(service, sheet_id: str, fila_id: int, bot: str, mensaje: str):
    """append() anade la fila en el servidor: dos bots simultaneos generan
    dos filas, nunca una perdida.
    NUNCA uses values.update sobre el rango entero."""
    service.spreadsheets().values().append(
        spreadsheetId=sheet_id,
        range="chat!A:D",
        valueInputOption="RAW",
        insertDataOption="INSERT_ROWS",
        body={"values": [[fila_id, bot, mensaje, ahora_utc()]]},
    ).execute()

```

14. Checklist de acuerdo con el compañero

Antes de escribir una sola línea de código, ambos equipos deben confirmar por escrito cada uno de estos puntos.

- [] El histórico es un **Google Sheet nativo**, no un `.xlsx`.
- [] `GOOGLE_SHEET_ID` compartido y ambos con permiso de **Editor**.
- [] Cada uno ha creado su Service Account y ha compartido el Sheet con su email.
- [] Hoja llamada chat, cabecera literal `id | bot | mensaje | timestamp`.
- [] Identificadores exactos: `claude_bot` y `codex_bot`.
- [] Timestamps en **UTC**, formato `YYYY-MM-DDTHH:MM:SSZ`.
- [] Escritura solo con `va.lues.append`. Prohibido reescribir el archivo.
- [] Regla de turno: escribir solo si la última fila **no** es propia.
- [] `claude_bot` escribe siempre la fila `id=1`; `codex_bot` nunca inicia.
- [] 16 filas totales. La 15 anuncia despedida, la 16 la responde.
- [] Mensajes de máximo 500 caracteres.

15. Cronograma

Fase	Tarea	Tiempo
Acuerdo	Revisar esta spec con el compañero y firmar el checklist	30 min
Preparación	Crear Sheet + Service Accounts + permisos	1 h
Preparación	Escribir <code>mi_informacion.txt</code>	30 min
Desarrollo	<code>config</code> , <code>knowledge</code> , <code>conversation</code> + pruebas unitarias	2 h
Desarrollo	<code>sheet_client</code> con reintentos	1,5 h
Desarrollo	<code>responder</code> + <code>validator</code>	1,5 h
Desarrollo	<code>main</code> (bucle) + registro de eventos	1 h
Pruebas	<code>bot_simulador</code> y conversación completa en local	1 h
Pruebas	E2E real con el bot del compañero	1 h
Cierre	Documentar resultados	30 min
	Total estimado	10–11 h

16. Preguntas frecuentes

¿Pueden los dos bots responder a la vez?

No. La regla de turno lo impide: un bot solo escribe si la última fila no es suya, así que en todo momento solo uno tiene derecho a escribir. Y aunque coincidieran por latencia, `va.lues.append` es atómico: se añadirían dos filas, pero no se perdería ningún mensaje.

¿Por qué no usamos .xlsx como decía la v1?

Un .xlsx es un archivo binario: para añadir una fila hay que descargarlo, modificarlo y volver a subirlo entero. Si el otro bot sube su versión en ese intervalo, tu mensaje desaparece sin dejar rastro. Un Sheet nativo permite añadir filas en el servidor, sin descargar nada.

¿Y si necesitamos entregar un Excel?

Google Sheets exporta a .xlsx con un clic (Archivo > Descargar > Microsoft Excel), o mediante la API al terminar. El requisito de entrega se cumple igual.

¿Cómo evito que el bot invente información?

Tres capas: prompt restrictivo con la base de datos inyectada, camino explícito de «no lo sé», y verificación posterior opcional con un modelo barato. Ver sección 8.1.

¿Puedo cambiar el modelo de Claude?

Sí, en el parámetro `model`. `claude-sonnet-5` o `claude-haiku-4-5` funcionan y son más baratos; para una charla casual la diferencia de calidad es pequeña.

¿Puedo cambiar el intervalo de 30 segundos?

Sí, `INTERVALO_LECTURA` en `.env`. Bajarlo acelera la conversación pero consume más cuota de la API de Sheets. No bajes de 5 segundos.

¿Qué pasa si Google Drive falla?

Reintentos con retroceso exponencial hasta 5 veces, y si no, se espera un ciclo completo y se vuelve a intentar. El bot no termina por un fallo transitorio.

¿Qué pasa si reinicio el bot a mitad de conversación?

`state.json` guarda el último `id` procesado, así que al arrancar no vuelve a responder a un mensaje que ya contestó.

Documenta cualquier anomalía y comunícate con tu compañero.